Build Task — Tool Library — Hints & Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper LangChain/Python). Read Basic first even if you already know this — it's the fastest way to spot exactly what Intermediate adds.

- Hint 1 — What you're building, and the exact pieces
- Hint 2 — The plan, and almost the whole thing
- Solution

Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 1 — What you're building, and the exact pieces

Basic Version

You're building three small files that work together: `tools.py` holds the tools, `tool_harness.py` runs one prompt against all of them, and `test_prompts.py` runs the Test Cases.

You need 3 kinds of tool: one that's pure logic (`add`), one that calls a real service (`get_weather`), and one built to fail on purpose (`flaky_lookup`). Every piece is something an exercise above already practiced:

- `add` and the `for call in response.tool_calls` round trip — the Basic exercise.
- `get_weather` , `build_weather_params()` and Doc02's `request_with_retry("GET", ...)` — the Real-world exercise.
- A Pydantic model per tool with `@tool(args_schema=...)` , and catching `ValidationError` — the Edge cases exercise, Approach 2.
- `return f"Error: {e}"` instead of a crash, and sending the result back with a `{"role": "tool", ...}` message — the Failure exercise.

Copy in unchanged first: `http_client.py` , `exceptions.py` , `logging_setup.py` from Doc02's Build Task.

Back to the Build Task · Hint 1 · Hint 2 · Solution

Intermediate Version

- **`tools.py`** — `AddArgs`, `WeatherArgs`, `LookupArgs` (Pydantic), the three `@tool(args_schema=...)` functions, and `ALL_TOOLS = [add, get_weather, flaky_lookup]`, so later documents import one list.
- **`flaky_lookup(query, should_fail=False)`** — `should_fail` makes the failure predictable for testing instead of random. Its description tells the model when to set it.
- **`tool_harness.py`** — a `ToolCallResult` dataclass (prompt, tool name, arguments, output, the model's final answer), `find_tool()`, `run_tool()` that turns a `ValidationError` into an error string, and `run_with_tools(prompt, tools)`, which runs every requested call and sends each result back before asking the model for its final answer.
- **`test_prompts.py`** — one prompt per Test Cases row, plus one for the real service.

Difference between Basic and Intermediate: Basic names the pieces and where each was practiced. Intermediate gives each file its exact shape.

Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
tools.py:
    a Pydantic args model per tool
    add, get_weather, flaky_lookup — each with @tool(args_schema=...)
    ALL_TOOLS = [add, get_weather, flaky_lookup]

tool_harness.py:
    ToolCallResult: prompt, tool_name, arguments, output, final_answer
    run_with_tools(prompt, tools):
        ask the model with the tools bound
        no tool call -> result with tool_name None
        else: run each call, send each result back, ask again

test_prompts.py:
    for each (label, prompt): run it, print tool, output, answer
```

Back to the Build Task · Hint 1 · Hint 2 · Solution

Intermediate Version

```
# tool_harness.py — the part that's new here
def run_tool(tools: list[BaseTool], call: dict) -> str:
    matching_tool = find_tool(tools, call["name"])
    if matching_tool is None:
        return "Error: tool not found"
    try:
        return str(matching_tool.invoke(call["args"]))
    except ValidationError:
        return "Error: the arguments failed the tool's checks."
```

Then `run_with_tools()` is the Failure exercise's round trip: append the model's response to `messages`, loop over `response.tool_calls`, append one tool message per call, and invoke again. Finish it, then compare against the Solution.

Difference between Basic and Intermediate: Basic is the plan. Intermediate is the one new function, almost complete.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Solution

Copied in unchanged: `http_client.py`, `exceptions.py`, `logging_setup.py` (Doc02's Build Task). Run everything from inside `practice/build_task/`.

Basic Version

Approach 1 — one flat harness, no round trip

Story — `tools.py`: the one library every later document (Doc07 onward) imports. Three kinds on purpose — pure logic, a real service, a tool that fails on purpose — each with a Pydantic `args_schema`, so bad arguments are rejected before the tool runs. **If not:** every later document would invent its own tools, none tested against this document's lessons.

```
# tools.py
from langchain_core.tools import tool
from pydantic import BaseModel, Field

from exceptions import PermanentHTTPError, TransientHTTPError
from http_client import request_with_retry

WEATHER_URL = "https://api.open-meteo.com/v1/forecast"

CITY_COORDINATES = {
    "lahore": {"latitude": 31.55, "longitude": 74.34},
    "karachi": {"latitude": 24.86, "longitude": 67.01},
}


# why: one Pydantic model per tool — the arguments are checked before
# the tool's body runs (the Edge cases exercise's args_schema)
class AddArgs(BaseModel):
    a: int
    b: int


class WeatherArgs(BaseModel):
    city: str = Field(min_length=1)
```

```
class LookupArgs(BaseModel):
    query: str = Field(min_length=1)
    should_fail: bool = False


@tool(args_schema=AddArgs)
def add(a: int, b: int) -> int:
    """Add two integers together and return the sum."""
    return a + b


def build_weather_params(city: str) -> dict | None:
    # how: the Real-world exercise's function, copied unchanged
    coords = CITY_COORDINATES.get(city.lower())
    if coords is None:
        return None
    return {
        "latitude": coords["latitude"],
        "longitude": coords["longitude"],
        "current_weather": True,
    }


@tool(args_schema=WeatherArgs)
def get_weather(city: str) -> str:
    """Get the current temperature for a named city."""
    params = build_weather_params(city)
    if params is None:
        return f"No coordinates on file for {city}."

    try:
        response = request_with_retry("GET", WEATHER_URL, params=params)
    except (TransientHTTPError, PermanentHTTPError) as e:
        return f"Error: the weather service failed: {e}"

    temperature = response["current_weather"]["temperature"]
    return f"Currently {temperature}°C in {city}"


@tool(args_schema=LookupArgs)
def flaky_lookup(query: str, should_fail: bool = False) -> str:
    """Look up information about a query. Set should_fail to true only
    when the user asks to force a failure."""
    # why: should_fail makes the failure controllable and predictable for
    # testing — not something you have to hope happens on its own.
    try:
        if should_fail:
```

```
            raise RuntimeError("simulated failure")
        return f"Result for {query}"
    except RuntimeError as e:
        return f"Error: {e}"


ALL_TOOLS = [add, get_weather, flaky_lookup]
```

Story — `tool_harness.py` (Basic): the Basic exercise's round trip, made reusable: bind the tools, ask, run the first requested tool. **If not:** Doc07's agent loop would have to build this from scratch.

```
# tool_harness.py
from dataclasses import dataclass

from langchain_openai import ChatOpenAI


@dataclass
class ToolCallResult:
    prompt: str
    tool_name: str | None
    arguments: dict
    output: str


def run_with_tools(prompt: str, tools: list) -> ToolCallResult:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    model_with_tools = model.bind_tools(tools)
    response = model_with_tools.invoke(prompt)

    if not response.tool_calls:
        return ToolCallResult(prompt, None, {}, response.content)

    call = response.tool_calls[0]
    for candidate in tools:
        if candidate.name == call["name"]:
            output = str(candidate.invoke(call["args"]))
            return ToolCallResult(prompt, call["name"], call["args"], output)

    return ToolCallResult(prompt, call["name"], call["args"], "Error: no tool")
```

```
# test_prompts.py
from tool_harness import run_with_tools
from tools import ALL_TOOLS
```

```
TEST_PROMPTS = [
    "what's 5 + 7?",
    "what's the weather in Lahore?",
    "tell me a joke",
    "look up the refund policy, and force the lookup to fail",
]

for prompt in TEST_PROMPTS:
    result = run_with_tools(prompt, ALL_TOOLS)
    print(result)
```

Expected output (example — shown wrapped):

```
ToolCallResult(prompt="what's 5 + 7?", tool_name='add',
    arguments={'a': 5, 'b': 7}, output='12')
ToolCallResult(prompt="what's the weather in Lahore?",
    tool_name='get_weather', arguments={'city': 'Lahore'},
    output='Currently 34.2°C in Lahore')
ToolCallResult(prompt='tell me a joke', tool_name=None, arguments={},
    output='Why did the chicken cross the road? ...')
ToolCallResult(prompt='look up the refund policy, and force the lookup
    to fail', tool_name='flaky_lookup', arguments={'query': 'refund
    policy', 'should_fail': True}, output='Error: simulated failure')
```

This works for these four prompts. But the model never sees the tool's result, so there's no final answer to check against "doesn't make up a fake success". And a model that sends an empty city makes `args_schema` raise a `ValidationError` that crashes the whole run — the "missing argument" Test Case isn't handled.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — every result sent back, bad arguments caught, and logged

`tools.py` is the same as the Basic version above.

Story — `tool_harness.py`: `run_tool()` turns bad arguments into an error string (Edge cases), and `run_with_tools()` sends every result back tagged with its `tool_call_id` and asks the model again (Failure) — so `final_answer` shows how the model really reacted. Every pick is logged with Doc01's `get_logger()`. **If not:** a failing tool's error would never reach the model, and one bad argument would crash the whole test run.

```
# tool_harness.py
from dataclasses import dataclass

# how: BaseTool is the type every @tool function has (the Failure
# exercise's count_tool_calls) — list[BaseTool] means "a list of tools"
from langchain_core.tools import BaseTool
from langchain_openai import ChatOpenAI
from pydantic import ValidationError

from logging_setup import get_logger


@dataclass
class ToolCallResult:
    prompt: str
    tool_name: str | None
    arguments: dict
    output: str
    final_answer: str


def find_tool(tools: list[BaseTool], name: str) -> BaseTool | None:
    # why: pulled out on its own so it's testable with plain tool objects
    # and a name string — no model call needed to prove it works.
    for candidate in tools:
        if candidate.name == name:
            return candidate
    return None
```

```
def run_tool(tools: list[BaseTool], call: dict) -> str:
    matching_tool = find_tool(tools, call["name"])
    if matching_tool is None:
        return "Error: tool not found"
    try:
        return str(matching_tool.invoke(call["args"]))
    except ValidationError:
        # why: bad arguments (an empty city) become a message the model
        # can read and react to — the Edge cases exercise's rule
        return "Error: the arguments failed the tool's checks."


def run_with_tools(prompt: str, tools: list[BaseTool]) -> ToolCallResult:
    logger = get_logger(__name__)
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    model_with_tools = model.bind_tools(tools)

    messages = [{"role": "user", "content": prompt}]
    response = model_with_tools.invoke(messages)

    if not response.tool_calls:
        # when: the model answered directly — a real, checkable outcome
        logger.info(f"no tool for: {prompt}")
        return ToolCallResult(prompt, None, {}, "", response.content)

    # how: the Failure exercise's round trip — the request, then one
    # tool message per call, then a second model call
    messages.append(response)
    outputs = []
    for call in response.tool_calls:
        logger.info(f"model picked {call['name']} with {call['args']}")
        output = run_tool(tools, call)
        outputs.append(output)
        messages.append(
            {"role": "tool", "content": output, "tool_call_id": call["id"]}
        )

    final = model_with_tools.invoke(messages)
    first_call = response.tool_calls[0]
    return ToolCallResult(
        prompt=prompt,
        tool_name=first_call["name"],
        arguments=first_call["args"],
        output=outputs[0],
        final_answer=final.content,
    )
```

Story — `test_prompts.py`: this Build Task's test file — one prompt per row of the Test Cases table, plus the real service, each printed on four short lines. **If not:** you'd ship the library never having watched it handle a missing argument or a failing tool.

```
# test_prompts.py
# Runs the 4 Test Cases from the README, plus the real-service tool.
# Run it from inside this folder: python test_prompts.py
# Makes real API calls — needs a valid OPENAI_API_KEY in .env.
from tool_harness import run_with_tools
from tools import ALL_TOOLS

# how: (label, prompt) pairs — one per row of the Test Cases table
TEST_PROMPTS = [
    ("clear match", "what's 5 + 7?"),
    ("real service", "what's the weather in Lahore?"),
    ("no tool", "tell me a joke"),
    ("missing argument", "what's the weather like?"),
    ("tool fails", "look up the refund policy, and force the lookup to fail"),
]


def main() -> None:
    for label, prompt in TEST_PROMPTS:
        result = run_with_tools(prompt, ALL_TOOLS)
        print(f"{label}: {prompt}")
        print(f"  tool:   {result.tool_name} {result.arguments}")
        print(f"  output: {result.output}")
        print(f"  answer: {result.final_answer}")


if __name__ == "__main__":
    main()
```

Expected output (model wording varies):

```
clear match: what's 5 + 7?
  tool:   add {'a': 5, 'b': 7}
  output: 12
  answer: 5 + 7 is 12.
real service: what's the weather in Lahore?
  tool:   get_weather {'city': 'Lahore'}
  output: Currently 34.2°C in Lahore
  answer: It's currently 34.2°C in Lahore.
no tool: tell me a joke
  tool:   None {}
  output:
```

```
  answer: Why did the chicken cross the road? To get to the other side.
missing argument: what's the weather like?
  tool:   None {}
  output:
  answer: Which city would you like the weather for?
tool fails: look up the refund policy, and force the lookup to fail
  tool:   flaky_lookup {'query': 'refund policy', 'should_fail': True}
  output: Error: simulated failure
  answer: The lookup failed, so I couldn't get the refund policy.
```

For "missing argument" the model usually asks for the city. If it sends an empty city instead, you'll see `tool: get_weather {'city': ''}` and `output: Error: the arguments failed the tool's checks.` — both are a pass for that Test Case.

Difference from Basic: bad arguments are caught instead of crashing the run, every tool result is sent back so the model's real final answer is recorded, all tool calls in a response get an answer (not just the first), and every pick is logged.

Which one should you actually use? Intermediate — it meets every requirement and every Test Case. Doc07's agent loop is this same round trip, repeated in a `while` loop until the model stops asking for tools.